

-- HW04

-- Doan Cong Thanh

-- 22125096

Question 1

(i) To consider whether or not the schedule is conflict-serializable, I will apply the Precedence graph for each schedule and find if it is acyclic.

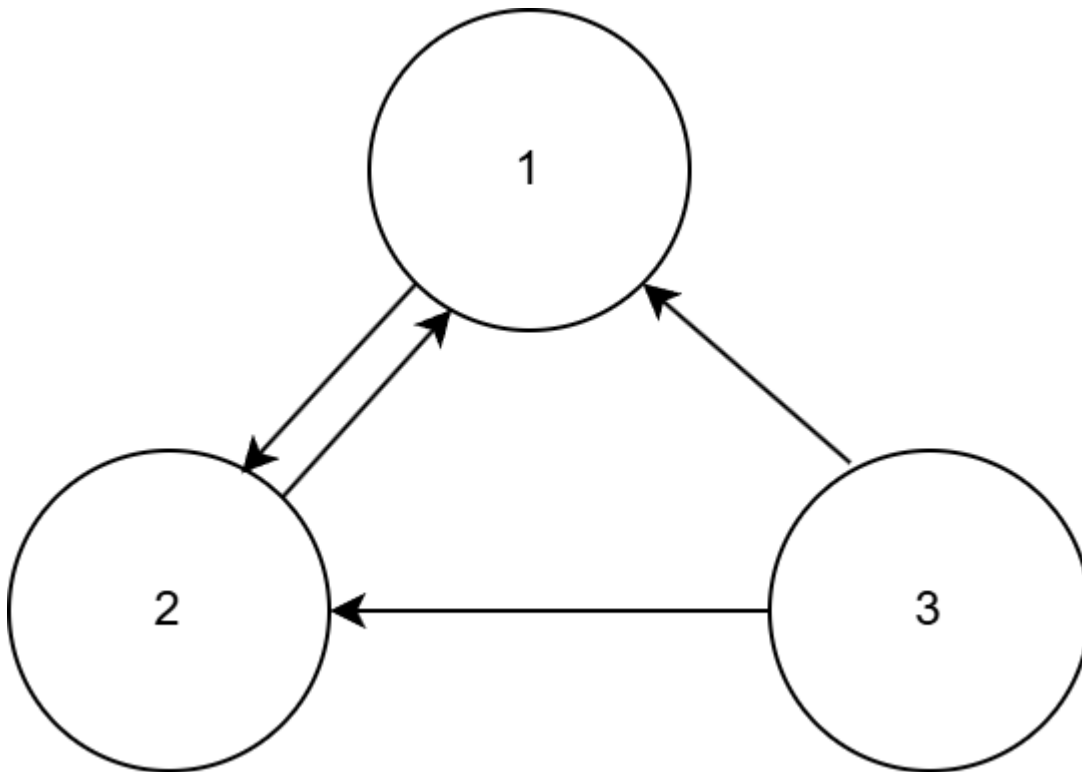
(a) $w_3(B)$; $w_3(A)$; $w_1(B)$; $r_2(B)$; $r_2(A)$; $w_1(A)$;

Since $w_3(B)$ is ahead of $w_1(B)$ and $r_2(B)$ in $S \rightarrow T_3 <_s T_1$ and $T_3 <_s T_2$

Since $w_1(B)$ is ahead of $r_2(B)$ in $S \rightarrow T_1 <_s T_2$

Since $r_2(A)$ is ahead of $w_1(A)$ in $S \rightarrow T_2 <_s T_1$

We have the graph:



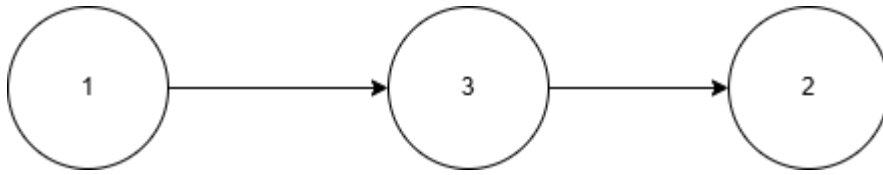
There is a loop in this graph $\rightarrow$ Cyclic $\rightarrow$ Not conflict-serializable

(b) $r_1(A)$; $r_2(B)$; $w_1(C)$; $r_3(B)$; $r_3(C)$; $w_2(B)$; $w_3(A)$;

Since $r_1(A)$ is ahead of $w_3(A)$ in $S \rightarrow T_1 <_s T_3$

Since $r_3(B)$ is ahead of $w_2(B)$ in $S \rightarrow T_3 <_s T_2$

We have the graph:



There is no loop in this graph $\rightarrow$ Acyclic $\rightarrow$ Conflict-serializable

(ii)

(a) $S = w_3(B); w_3(A); w_1(B); r_2(B); r_2(A); w_1(A);$

Since S is not conflict-serializable $\rightarrow$ There is no conflict-equivalent serial schedule

(b) $S = r_1(A); r_2(B); w_1(C); r_3(B); r_3(C); w_2(B); w_3(A);$

The conflict - equivalent serial order is: $1 \rightarrow 3 \rightarrow 2$

Serial schedule: $r_1(A); w_1(C); r_3(B); r_3(C); w_3(A); r_2(B); w_2(B);$

Question 2

(a) $S_1 = w_1(A); w_2(B); w_3(C); c_1; r_3(A); c_2; r_3(B); c_3;$

Determine pairs of (T_j, T_i) which means T_j reads from T_i

Since $w_1(A) <_s r_3(A)$ and there is no $a_1 <_s r_3(A) \rightarrow T_3$ reads from T_1

Since $w_2(B) <_s r_3(B)$ and there is no $a_2 <_s r_3(B) \rightarrow T_3$ reads from T_2

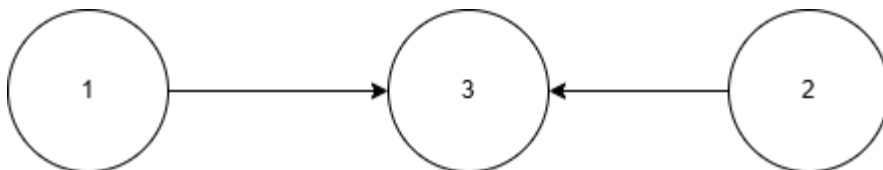
Since T_3 reads from T_1 and $c_1 <_s c_3$, T_3 reads from T_2 and $c_2 <_s c_3$

$\Rightarrow S_1$ is recoverable

Since $w_1(A)$ is ahead of $r_3(A)$ in $S_1 \rightarrow T_1 <_s T_3$

Since $w_2(B)$ is ahead of $r_3(B)$ in $S_1 \rightarrow T_2 <_s T_3$

We have the graph



Since there is no loop in this graph $\rightarrow$ Acyclic $\rightarrow$ Conflict-serializable

(b) $S_2 = r_1(A); w_2(A); w_2(B); c_2; r_1(B); c_1;$

Determine pairs of (T_j, T_i) which means T_j reads from T_i

Since $w_2(B) <_s r_1(B)$ and there is no $a_2 <_s r_1(B) \rightarrow T_1$ reads from T_2

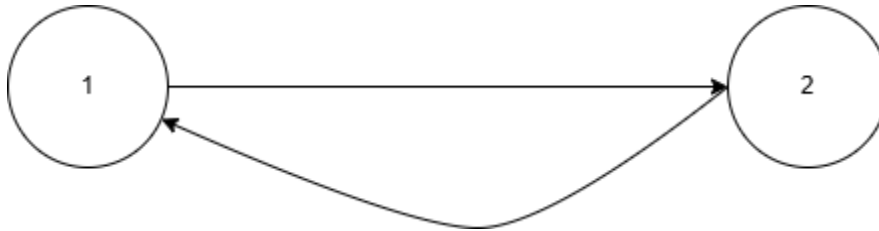
Since T_1 reads from T_2 and $c_2 <_s c_1$

⇒ S2 is recoverable

Since $r1(A)$ is ahead of $w2(A)$ in S2 → $T1 <_s T2$

Since $w2(B)$ is ahead of $r1(B)$ in S2 → $T2 <_s T1$

We have the graph:



Since there is a loop (1→2,2→1) → Cyclic → Not conflict-serializable

(c) S3 = $w1(A); w1(B); r2(A); w2(B); a2; w1(C); c1; r3(B); w3(C); c3;$

Determine pairs of (T_j, T_i) which means T_j reads from T_i

Since $w1(A) <_s r2(A)$ and there is no $a1 <_s r2(A)$ → T_2 reads from T_1

Since $w1(B) <_s r3(B)$ and there is $w1(B) <_s w2(B) <_s r3(B)$ and $a2 <_s r3(B)$ → T_3 reads from T_1

Since T_3 reads from T_1 and $c_1 <_s c_3$

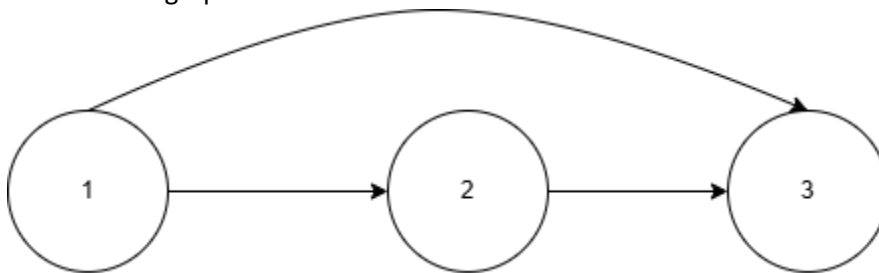
⇒ S3 is unrecoverable

Since $w1(A)$ is ahead of $r2(A)$ in S → $T1 <_s T2$

Since $w1(B)$ is ahead of $r3(B)$ in S → $T1 <_s T3$

Since $w2(B)$ is ahead of $r3(B)$ in S → $T2 <_s T3$

We have the graph:



Since there is no loop in this graph → Acyclic → Conflict-serializable

Question 3

(a) $T1 = L1(C); w1(C); L1(A); r1(A); w1(A); L1(B); r1(B); w1(B); Commit; U1(A); U1(C); U1(B);$

Since there is no unlock operation before $L1(B)$, which is the last lock operation in the transaction and there is no existence of lock operation after $U1(A)$, which is the first unlock action in the transaction.

→ T1 follows two-phase locking

T2 = L2(B); r2(B); w2(B); L2(A); r2(A); w2(A); Commit; U2(A); U2(B);

Since there is no unlock operation before L2(A), which is the last lock operation in the transaction and there is no existence of lock operation after U2(A), which is the first unlock action in the transaction.

→ T2 follows two-phase locking

(b)

T1	T2
L1(C)	
w1(C)	
	L2(B)
	r2(B)
	w2(B)
	L2(A)
	r2(A)
	w2(A)
	Commit
	U2(A)
	U2(B);
L1(A)	
r1(A)	
w1(A)	
L1(B)	
r1(B)	
w1(B)	
Commit	
U1(A)	
U1(C)	
U1(B);	

This schedule satisfies:

- Consistency of transactions
- Legality of schedules
- Two-phase locked transactions

→ S will be converted to a conflict-equivalent serial schedule

→ S is conflict-serializable

(c)

T1	T2
L1(C)	
w1(C)	
	L2(B)
	r2(B)
	w2(B)
	L2(A)

r2(A)
 w2(A)
 Commit
 U2(A)
 U2(B);

L1(A)
 r1(A)
 w1(A)
 L1(B)
 r1(B)
 w1(B)
 Commit
 U1(A)
 U1(C)
 U1(B);

Determine pairs of (T_j, T_i) which means T_j reads from T_i

Since $w2(B) <_s r1(B)$ and there is no $a2 <_s r1(B) \rightarrow T1$ reads from $T2$

Since $T1$ reads from $T2$ and $c_2 <_s c_1$ in S

$\rightarrow$ This schedule S is recoverable

(d)

T1
 L1(C)
 w1(C)

T2

L2(B)
 r2(B)
 w2(B)
 L2(A)
 r2(A)
 w2(A)
 Commit
 U2(A)
 U2(B);

L1(A)
 r1(A)
 w1(A)
 L1(B)
 r1(B)
 w1(B)
 Commit
 U1(A)
 U1(C)
 U1(B);

Determine pairs of (T_j, T_i) which means T_j reads from T_i

Since $w2(B) <_s r1(B)$ and there is no $a2 <_s r1(B) \rightarrow T1$ reads from $T2$

Since $T1$ reads from $T2$, and $T2$ is completed before transaction $T1$ reads value.

$\rightarrow$ This schedule S avoids cascading rollback

(e) Yes, these annotated transactions can result in a deadlock

T1	T2
L1(C)	
w1(C)	
	L2(B)
	r2(B)
	w2(B)
L1(A)	
r1(A)	
w1(A)	
	L2(A)
L1(B)	
r1(B)	
w1(B)	
Commit	
U1(A)	r2(A)
U1(C)	w2(A)
U1(B);	Commit
	U2(A)
	U2(B);

From this schedule. Firstly, the transaction $T1$ will request a lock on element C , then $T2$ locks on B and performs some operations. Move on, $T1$ locks on A . And when $T2$ request another lock on A (which is occupied by $T1$), all of the actions after this one will be denied and wait until $T1$ unlock A . However, when we execute actions in $T1$, it asks for a lock on B (which is used by $T2$) $\rightarrow$ Deadlock problem occurs.

Question 4

(a) The system uses UNDO logging only

<T1, start>	<T2, commit>: T2 is completed,
<T1, X, 10>	do not undo this transaction.
<T2, start>	T1 is incomplete, undo this
<T2, Z, 12>	transaction.
<T2, Y, 20>	Actions performed by the
<T2, commit>	recovery manager:
	- Undo T1:
	+ Write(X,10)
	+ Abort(T1)

(b) The system uses REDO logging only

<T1, start>	<T2, commit>: T2 is completed,
<T1, X, 10>	redo this transaction.
<T2, start>	T1 is incomplete, do not redo
<T2, Z, 12>	this transaction.
<T2, Y, 20>	Actions performed by the
<T2, commit>	recovery manager:
	- Redo T2:
	+ Write(Z,12)
	+ Write(Y,20)
	+ Abort(T1)

(c) The system uses UNDO/REDO logging with checkpoint

<T1, start>	<checkpoint>: The recovery
<T1, X, 10, 11>	manager just performs action
<T1, commit>	from bottom upward until find
<checkpoint>	the checkpoint.
<T2, start>	T2 and T4 are incomplete, undo
<T2, Y, 20, 21>	these transactions, do not redo
<T3, start>	these ones.
<T3, Z, 30, 31>	<T3, commit>: T3 is complete,
<T4, start>	redo this transaction, do not
<T3, commit>	undo this one.
<T4, W, 40, 41>	Actions performed by the
<T2, Z, 80, 30>	recovery manager:
	- Redo:
	+ Write(Z,31)
	- Undo:
	+ Write(Z,80)
	+ Write(W,40)
	+ Write(Y,20)
	+ Abort(T2)
	+ Abort(T4)